

Introduction

Some interview problems test whether you know a data structure. This one tests something different: whether you can resist the urge to simulate a problem and instead find the invariant that makes simulation unnecessary.

LeetCode 2029, Sum Game, looks like a game theory problem, and on the surface it is. Two players, Alice and Bob, alternate turns filling in missing digits of a string, and one of them wants two halves to sum equally while the other wants them to differ. The input can be up to 100,000 characters long, which immediately rules out trying every possible sequence of moves. That constraint is the real hint: the problem wants you to notice a mathematical shortcut, not build a game tree.

This is exactly the kind of problem interviewers like, because it separates candidates who pattern-match ("this smells like a Stone Game problem, so I need value optimization") from candidates who actually trace through what each move does to the state. Once you see that every move can be perfectly cancelled by the opponent, the whole problem collapses into counting blanks and comparing sums. It's a great problem for practicing how to spot when a "game" is really just arithmetic in disguise.

Problem Overview

You're given a string `num` of even length, made up of digits `0-9` and the character `?`, which represents a blank digit. Alice and Bob take turns replacing a `?` with any digit from `0` to `9`, Alice moving first, continuing until every blank has been filled.

Once the string is complete, split it exactly in half. Add up the digits in the left half and the digits in the right half.

- **Bob wins** if the left sum equals the right sum.
- **Alice wins** if they differ.

Both players play optimally — Bob always tries to force equality, Alice always tries to prevent it. Given the string, determine who wins.

Constraints worth internalizing: - The string length is always even, so both halves are always the same size. - Length can reach 100,000, so any solution that tries multiple digit choices per blank is immediately too slow. - Every character is either a digit or `?` — no other symbols to handle.

Example

Take the string "25??".

- Left half: "25" → sum = 7
- Right half: "??" → both blank

Alice moves first. She places a 9 in one of the right blanks. Now the right half already totals 9 by itself — more than the left's 7 — with one blank still open for Bob.

Bob's move can't undo that gap. Whatever digit Bob places in the last blank, the right sum will be at least 9 (since digits are non-negative), and it can never be pulled back down to 7. Alice wins.

The reason this works: Alice had **both** blanks in one half. She placed the first one to create an unbeatable gap, and Bob had no blank of his own to counter with. That detail — who holds the blanks, not just how many exist — is the seed of the entire solution.

Intuition

The instinct on seeing "two players alternately choose values, one wants a sum equal, one wants it unequal" is to reach for a Stone Game-style approach: track possible outcomes, maybe minimax over remaining choices, maybe memoize by state. That's the wrong shape for this problem, and recognizing why is the whole exercise.

Stop thinking about two separate sums. Think about one number: $\text{diff} = \text{left_sum} - \text{right_sum}$. Bob wins if he can force diff to become exactly zero. Alice wins if she can keep it away from zero.

Now watch what happens once both halves still have blanks remaining. Say Alice fills a blank with some digit d , shifting diff by some amount. Bob's next move isn't a search — he can always pick a digit for one of his own blanks that exactly cancels Alice's shift. Digits only range from 0 to 9, and Bob has the freedom to choose any digit in that same range, so cancellation is guaranteed by the ten-digit alphabet itself, not something Bob has to calculate his way into.

That single observation converts the problem from **value optimization** ("what's the best digit to place?") into **pure counting** ("who runs out of moves to cancel first?"). As long as both halves have at least one blank left, any move Alice makes, Bob can neutralize. The game only gets decided by two static facts:

1. The current difference between the two sums (from digits already present).
2. The difference in blank counts between the two halves.

If the blank counts are equal, Bob can mirror every Alice move — pairing digits so each pair sums to 9 (since if Alice plays `d`, Bob plays `9 - d`, keeping the *pair's* contribution to `diff` fixed) — and Bob wins **only if** the existing difference is already zero.

If the blank counts are unequal, it gets more interesting. Naively you might think "whoever has more blanks wins for their opponent" — but that's not quite right either. Consider one blank on the left, three on the right. Two of the three right-side blanks can pair up with each other under Bob's control, both summing to 9 regardless of what Alice does with them. Only the one truly unpaired blank is a free, uncanceled move — and that's the one Alice can exploit.

This is the key realization: **only unpaired blanks matter**. Everything else cancels.

Brute Force Solution

Idea: Simulate the game directly. At each turn, try every digit 0–9 in every remaining blank, and recurse to see if the current player can force a win.

Algorithm: 1. Track the current string state and whose turn it is. 2. For each blank, try placing each digit 0–9. 3. Recurse into the resulting state. 4. Alice wins if any move leads to a state where she wins (from her perspective); Bob wins if any move leads to a state where the sums end up equal. 5. Use memoization on the state to avoid recomputation.

Advantages: - Conceptually simple, directly matches the problem statement. - No missed edge cases — it always mirrors truly optimal play by construction.

Disadvantages: - Explodes combinatorially. Even 6 blanks means up to 10^6 digit-assignment branches. - With strings up to 100,000 characters (up to 50,000 blanks), this is astronomically infeasible. - Memoization doesn't save you either — the state space (partially filled strings) is enormous.

Complexity: Time is exponential, roughly $O(10^k)$ where k is the blank count. Space is similarly explosive from memoization tables. Neither is usable beyond toy inputs.

Optimal Solution

Instead of simulating, extract the two quantities that actually determine the outcome:

1. `diff` — the sum of known digits in the left half minus the sum of known digits in the right half.
2. `leftBlanks` and `rightBlanks` — the count of `?` in each half.

Step-by-step reasoning:

Case	Outcome
Total blanks (<code>leftBlanks + rightBlanks</code>) is odd	Alice wins outright — one move is always left uncanceled
Total blanks is even, and <code>diff == 9 * (leftBlanks - rightBlanks) / 2</code>	Bob wins
Total blanks is even, and <code>diff != 9 * (leftBlanks - rightBlanks) / 2</code>	Alice wins

Why `9 * (leftBlanks - rightBlanks) / 2`? Suppose the left half has more blanks than the right, by some amount `g = leftBlanks - rightBlanks`. Pairs of blanks across both halves cancel via Bob's 9-complement trick, so only `g` blanks on the left side go permanently unmatched. Whoever fills those unmatched blanks (functionally, the side with excess blanks) can push the left sum up by as much as `9` per blank, so the maximum swing Bob can be forced to absorb — while still holding the difference at zero — is `9 * g`. When `g` is even, that swing splits evenly into "half attacking, half defending" pairs, giving the `/2` factor. Bob can only force a tie if the pre-existing digit `diff` already sits exactly at that forced swing.

If total blanks are odd, no matter how the pairing works out, one blank is always structurally unpaired — because 2 blanks (one per player) is required for a cancel, and any leftover single blank hands its owner (Alice, since Bob can never guarantee cornering the last unmatched move) full control to push the difference off zero.

Python Code

```
class Solution:
    def sumGame(self, num: str) -> bool:
        n = len(num)
        half = n // 2
        left, right = num[:half], num[half:]

        left_blanks = left.count('?')
        right_blanks = right.count('?')
        left_sum = sum(int(c) for c in left if c != '?')
        right_sum = sum(int(c) for c in right if c != '?')

        total_blanks = left_blanks + right_blanks
        if total_blanks % 2 == 1:
            return True # Alice wins: an unpaired move always exists

        diff = left_sum - right_sum
        blank_gap = left_blanks - right_blanks

        return diff != 9 * blank_gap // 2
```

Code Walkthrough

- `half = n // 2` — the midpoint, since the string length is guaranteed even.
- `left, right = num[:half], num[half:]` — split into the two halves being compared.
- `left_blanks, right_blanks` — count of `?` per half using the built-in `str.count`.
- `left_sum, right_sum` — sum of the digits that are already fixed, skipping blanks entirely, since unfilled positions contribute nothing until chosen.
- `total_blanks % 2 == 1` — the odd-blank check. If true, return `True` immediately (Alice wins) without even computing sums.
- `diff = left_sum - right_sum` — the existing imbalance before any blanks are filled.
- `blank_gap = left_blanks - right_blanks` — how lopsided the blank distribution is.
- Final return — Bob only wins (function returns `False`) when the current `diff` exactly equals the forced swing `9 * blank_gap // 2`; otherwise Alice wins.

Dry Run

Input: "25??"

1. `n = 4, half = 2`
2. `left = "25", right = "??"`
3. `left_blanks = 0, right_blanks = 2`
4. `left_sum = 2 + 5 = 7, right_sum = 0`
5. `total_blanks = 0 + 2 = 2` → even, continue
6. `diff = 7 - 0 = 7`
7. `blank_gap = 0 - 2 = -2`
8. `9 * (-2) // 2 = -9`
9. Is `diff (7) != -9`? Yes.
10. Return `True` → Alice wins.

This matches the earlier hand-traced example, where Alice used her two blanks in the right half to blow past the left's fixed sum of 7.

Complexity Analysis

Time: $O(n)$. The solution does a constant number of passes over the string (slicing, counting `?`, summing digits) — no nested loops, no branching factor.

Space: $O(n)$ in this implementation, because Python string slicing (`num[:half]`) creates new string copies. This can be reduced to $O(1)$ by iterating over the original string once with an index-based loop instead of slicing, tracking left/right sums and blank counts directly without materializing substrings.

Both bounds are correct because the entire algorithm boils down to one linear scan and a handful of arithmetic comparisons — there's no case where more work is needed regardless of input contents.

Alternative Solutions

A minor variant avoids the two substring slices and computes everything in a single pass:

```
class Solution:
    def sumGame(self, num: str) -> bool:
        n = len(num)
        half = n // 2
        left_sum = right_sum = 0
        left_blanks = right_blanks = 0

        for i, c in enumerate(num):
            if c == '?':
                if i < half:
                    left_blanks += 1
                else:
                    right_blanks += 1
            else:
                if i < half:
                    left_sum += int(c)
                else:
                    right_sum += int(c)

        if (left_blanks + right_blanks) % 2 == 1:
            return True

        return (left_sum - right_sum) != 9 * (left_blanks - right_blanks) // 2
```

This trades the readability of slicing for true $O(1)$ auxiliary space, since no substrings are materialized. Functionally identical outcome, better for very large inputs where allocation matters.

Edge Cases

- **All digits, no blanks:** `total_blanks = 0`, even, `diff` is just the fixed sums' difference, `blank_gap = 0`. Bob wins only if the two halves already sum equally.
- **All blanks, even count:** e.g., `"????"` — both players fill freely, `blank_gap` could be zero if split evenly, and Bob wins by mirroring since starting `diff = 0`.

- **Single blank total (odd):** any string with exactly one `?` — returns `True` immediately, Alice wins, since that lone move can never be countered.
- **Minimum length (2 characters):** e.g., `"5?"` — one blank, odd total, Alice wins trivially.
- **Maximum length (100,000 characters):** the linear-time approach is essential here; anything super-linear risks timing out.
- **Both halves entirely blank with unequal counts** isn't possible since halves are always equal length — but blanks *within* equal-length halves can still differ freely, which is the actual source of asymmetry.

Common Mistakes

1. **Trying to simulate or brute-force the game** — works on tiny inputs, times out immediately on real constraints (up to 100,000 characters).
2. **Treating this as a value-optimization / minimax problem** like Stone Game, when the fixed 0–9 digit range makes it a pure counting problem instead.
3. **Assuming "more blanks always favors that side's opponent"** without accounting for pairing — a lopsided blank count can still fully cancel except for the genuinely unmatched excess.
4. **Forgetting the odd-total-blanks shortcut**, and instead trying to compute the general formula even when it doesn't apply — the odd case must be checked first and separately.
5. **Sign errors in `9 * blank_gap // 2`** — mixing up which half has more blanks, or forgetting the direction of `diff = left_sum - right_sum`, silently flips the winner.
6. **Not handling the case where blanks exist but the sums are already unequal** — some implementations mistakenly assume Bob only loses due to blank imbalance, ignoring pre-existing digit differences.

Interview Questions

- "How would you prove that Bob's cancellation strategy is always achievable, not just usually achievable?"
- "What changes if digits could range 0–99 instead of 0–9?" (Cancellation still works if the range is symmetric enough, but the constant `9` in the formula changes to reflect the new max digit.)
- "Can you reduce the space complexity to $O(1)$?" (Yes — avoid string slicing, iterate with an index.)
- "What's the general pattern here that applies to other alternating games?" (Track a single running difference and the move counts on each side; look for a cancellation strategy before assuming you need a search.)

- "How would you detect, programmatically, whether a game reduces to a parity argument versus genuinely requiring game-tree search?"

Similar Problems

- **LeetCode 486 - Predict the Winner:** Another two-player optimal-play problem, but here value optimization actually is required — a good contrast case for when minimax is necessary versus when it isn't.
- **LeetCode 877 - Stone Game:** The classic value-optimization alternating game; useful to compare against Sum Game to see why the digit-range constraint here avoids that complexity.
- **LeetCode 292 - Nim Game:** Another "collapses to one arithmetic check" game, reinforcing the pattern of finding an invariant instead of simulating.
- **LeetCode 1927 (the numbering echoed in the title) style parity problems:** general practice for problems where evenness/oddness of a count decides the entire outcome.

Key Takeaways

- Whenever a two-player alternating game involves choosing from a bounded, symmetric range of values, check first whether moves can be perfectly cancelled — that turns a search problem into a counting problem.
- Reduce two competing quantities (two sums) into one running difference. It's almost always easier to reason about a single number trending toward or away from a target.
- The parity of unpaired resources (blanks, moves, turns) is frequently the entire answer, not just a tiebreaker.
- Brute-force game simulation is a fine way to *verify* your intuition on small inputs, but should never be the shipped solution when constraints hint otherwise.

Watch the Video

For the full hand-traced walkthrough, including the "25??" quiz example worked out live and a side-by-side timing comparison between brute force and the optimal solution, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that works through LeetCode problems, systems concepts, and everyday engineering topics with an emphasis on building real intuition — not just memorizing a solution pattern. Each episode traces the reasoning an experienced

engineer would actually use to arrive at an answer, so the goal isn't just solving today's problem, but recognizing the underlying pattern the next time it shows up somewhere else.

Call To Action

If this walkthrough helped the parity trick click, drop a like on the video and subscribe on YouTube for more problems broken down this way. For deeper dives and behind-the-scenes notes on how these solutions get built, subscribe to the Substack newsletter. Got a different way you'd explain the cancellation argument, or a follow-up problem you want covered next? Leave a comment — and if you know someone prepping for interviews, share this one with them.

Quick Review

Alice/Bob remove digits from a number till 1 left each; who wins?

Both play optimally to make final numbers unequal (Alice) or equal (Bob) — think parity/pairing game.

Time complexity of the Sum Game solution?

$O(n)$, single pass counting knowns/unknowns and sums on each half.

Space complexity of Sum Game?

$O(1)$ extra — just running counts and sums, no auxiliary arrays.

Trickiest bug in Sum Game?

Forgetting moves cancel in pairs — only the parity of unknown-count difference between halves matters, not raw values.

What actually decides the winner in Sum Game?

Compare $(\text{sumKnown1} - \text{sumKnown2})$ against $9 * (\text{unknown2} - \text{unknown1}) / 2$ after accounting for parity of total unknowns.

Interview follow-up: why must total unknown count be even to favor Bob easily?

Even unknowns split evenly between players, letting Bob always mirror Alice's move to force equality.

Sum Game — free Python cheat sheet